HERD: Harness-Evolving Recursive Distillation for Continual Learning

Anonymous ACL submission

Abstract

Continual learning requires a model to keep acquiring new skills and knowledge without compromising its existing capabilities. Parametric methods suffer from catastrophic forgetting and loss of plasticity. Non-parametric methods, such as prompts, skills, and harnesses, leverage in-context learning, are limited by context length and lossy plain-text compression. To effectively combine their complementary strengths, we propose HERD (Harness-Evolving Recursive Distillation), a hybrid continual learning framework that learns efficiently in the harness and internalizes experience into cumulative capabilities via distillation. On the harness side, HERD continually refines its skills to capture reusable patterns, while retaining high-residual examples that the skills fail to capture in a bounded memory. In the model parameters, HERD performs drift-constrained distillation with dual anchors, substantially reducing forgetting over long recursive runs. On the 25-stage task stream of TRACE+17, under fixed memory and compute budgets, HERD matches the strongest parametric baseline in new-task acquisition while exhibiting the least forgetting and the strongest retention of general capabilities.

1 Introduction

Large language models (LLMs) are highly capable, yet cannot independently carry out much of the work people need done (Mazeika et al., 2025). One pressing problem in closing this gap is continual learning: humans keep learning from experience while retaining what they know, whereas models no longer change once deployed. Continual learning (CL) seeks to enable this ability by learning from a stream of tasks without forgetting existing capabilities or retraining from scratch on earlier data (McCloskey and Cohen, 1989; Wu et al., 2024).

Parametric methods, however, inevitably encounter catastrophic forgetting (Kirkpatrick et al., 2017; Huang et al., 2024) and loss of plasticity (Dohare et al., 2024; Yang et al., 2026). They also give little control over what is internalized: raw experience carries noise, duplicates, and conflicts alongside the few samples that matter.

In-context learning (ICL) offers a non-parametric path to continual learning (Brown et al., 2020; Gutiérrez et al., 2025). New knowledge is stored in external memory and retrieved into context, so a frozen LLM learns across tasks (Shi et al., 2025). This appears to sidestep the flaws above: a frozen model forgets nothing, loses no plasticity, and keeps knowledge inspectable and editable. It is also the norm in practice: users write prompts, skills, and harnesses rather than post-train, and recent works build continual learning systems on harnesses (Zhang et al., 2026c; Kang et al., 2026). Could a harness alone, recursively compressing its context, solve continual learning once models are strong and context windows long enough? We argue that it cannot. External stores can hold unbounded data, but nothing accumulates in the model itself: whatever it acts on must pass through the context, so experience is compressed into bounded plain text, which loses information at a very high rate. The more experience accumulates, the more is lost.

The resolution lies in the weights themselves. A few hundred gigabytes of parameters recall history, law, medicine, and science (Allen-Zhu and Li, 2025; Morris et al., 2026; Roberts et al., 2020), or even reproduce entire books nearly verbatim (Cooper et al., 2026). Language models are general-purpose compressors (Delétang et al., 2024), and compression is intelligence (Hutter, 2005). Continual learning thus still must rely on weight updates; what it must avoid is updating them on everything. Each layer should store what it is best suited for: capacity in the weights, selection in the harness.

We therefore propose HERD (Harness-Evolving Recursive Distillation), a hybrid continual learning

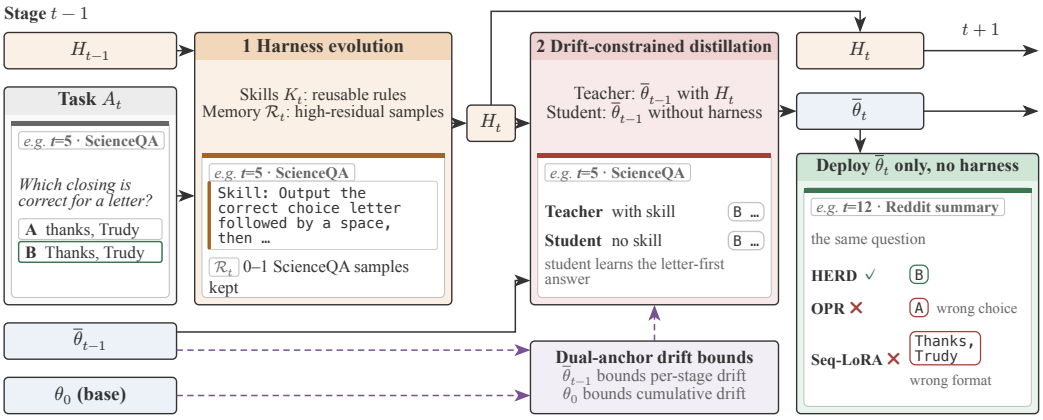


Figure 1: HERD overview: one recursive stage. The harness $H_t = (K_t, \mathcal{R}_t)$ evolves on task A_t and is distilled into the weights; drift budgets keep the student close to two anchors, the previous checkpoint $\bar{\theta}_{t-1}$ and the base θ_0. H_t and $\bar{\theta}_t$ recur, and only $\bar{\theta}_t$ is deployed. The *e.g.* tiles follow one ScienceQA item, learned at $t = 5$ and asked again at $t = 12$: two parametric baselines fail, OPR choosing the wrong option and Seq-LoRA writing the right text without the choice letter, while HERD answers from its weights alone across three seeds.

system in which the harness is the fast, selective learning layer and the weights are the slow, persistent layer. The design mirrors the human brain: a harness resembles episodic memory formed during the day, flexible but limited in capacity, which sleep consolidates into long-term memory (McClelland et al., 1995). HERD likewise internalizes its harness into the weights after every stage, under fixed memory and compute budgets and at no added inference cost. It does not eliminate forgetting, but preserves existing capabilities as far as possible while the model keeps learning.

HERD is designed around recursion: it must run stably stage after stage, which raises two challenges. First, the context is bounded, so the harness must decide what to change and what to keep. Second, recursive training drifts, and unchecked drift ends in collapse. HERD answers each with one module (Figure 1). For the first, the harness evolves with the task stream: the model iterates on its skills to abstract reusable rules, and retains in a bounded memory the high-residual samples the skills cannot cover. For the second, HERD performs drift-constrained distillation with dual anchors. The student is the previous checkpoint trained to match itself under the harness, which shifts the model less than SFT and thus forgets less (Shenfeld et al., 2026, 2025). The previous checkpoint bounds per-stage drift and the base model bounds cumulative drift. The harness is kept, inspectable and editable, for the next stage; only the weights are deployed.

Across a 25-stage stream HERD keeps more of what it learns than any other method: its final mean over all seen tasks is 0.706, above every parametric baseline and 27 points above the non-parametric ones, while its acquisition, 0.713, is statistically indistinguishable from the plasticity floor set by Seq-LoRA (Table 1). Forgetting is 0.010, about half or less that of every weight-updating baseline but SDFT, whose generation collapses (Appendix D); the general test falls 6.3 points where these other baselines lose 20.0 to 25.5. In Figure 1, the rule learned at stage 5 still holds in the weights alone at stage 12 across three seeds.

Our contributions are: (1) HERD, a hybrid continual learning system that evolves a harness of skills and high-residual memory and recursively internalizes it into the weights (§2); (2) drift-constrained distillation with dual anchors, where the previous checkpoint bounds per-stage drift and the base model bounds cumulative drift (§2.4); (3) an evaluation on TRACE+17, a 25-stage stream of eight TRACE tasks (Wang et al., 2023b) and 17 Super-NaturalInstructions tasks (Wang et al., 2022), against parametric, non-parametric, and hybrid baselines (§3).

Our code and dataset are anonymously available at `https://anonymous.4open.science/r/HERD-67AB/`.

2 Methodology

2.1 Preliminaries

Paradigms. We consider an ordered stream of tasks $\mathcal{S} = (\mathcal{T}_1, \ldots, \mathcal{T}_T)$, each with its own splits and a metric m_t in $[0, 1]$. Every example is an input

and a target text; any dataset-specific instruction is part of the example. The learner after stage t is $\mathcal{L}_t = (f_t, M_t)$: a prediction model and everything it retains across stages, external indexes included.

Problem formulation. Stage t exposes $E_t = (D_t^{\text{train}}, D_t^{\text{val}})$ once and updates $\mathcal{L}_t = A(\mathcal{L}_{t-1}, E_t)$ from $\mathcal{L}_0 = (f_{\theta_0}, M_0)$; earlier training sets are reachable only through M. Retained state and update compute are bounded, $\text{size}(M_t) \le B_M$ and $\sum_t \text{cost}(A, \mathcal{L}_{t-1}, E_t) \le B_C$, and any context attached at inference is reported with the scores. The learner is scored on a fixed schedule on the tasks seen so far and on fixed general-capability probes. With $R_{t,j}$ the score on task j after stage t and $\Delta C_k = C_{T,k} - C_{0,k}$ the change in the score on probe k over the stream, we report final average performance, forgetting, and the worst change in general capability,

$$\begin{aligned} \text{AA} &= \tfrac{1}{T}\textstyle\sum_j R_{T,j}, \quad \Delta C_{\min} = \min_k \Delta C_k, \\ \text{F} &= \tfrac{1}{T-1}\textstyle\sum_{j<T} \max(0,\, R_{j,j} - R_{T,j}), \end{aligned} \tag{1}$$

together with the mean change ΔC (§3.1). The objective is to maximize AA and minimize F without negative ΔC or $\Delta C_{\min}$, within B_M and B_C.

Harness representation. We use harness in the sense of DeepSeek Harness (DSH) (DeepSeek-AI, 2026) and of harness-level continual learning (Kang et al., 2026; Zhang et al., 2026a): the state and procedures around a model at inference, such as system prompts, skills, memories, and tools, as opposed to its weights. The harness of HERD is $H = (K, \mathcal{R})$. K is a registry of skills, each a routing prefix and instructions; an input x is routed by prefix, and the matched instructions $K(x)$ become the system message. $\mathcal{R}$ is a residual memory of whole training records, prompt and verified answer, kept verbatim. The budgets are B_K and B_R, with $B_M = B_K + B_R$. Executing the harness is $f(x; \theta, K)$: the model θ answering x under the system message $K(x)$.

2.2 HERD in One Stage

In this notation, HERD after stage t is $\mathcal{L}_t = (f_{\bar\theta_t}, (K_t, \mathcal{R}_t, G))$: merged dense weights $\bar\theta_t$, the harness $H_t = (K_t, \mathcal{R}_t)$, and a fixed set G of general probes reserved before the stream. Figure 1 shows one stage, which learns the task twice. First, the harness evolves on a bounded window of the task (§2.3). Second, the checkpoint, given the skill and the verified answer, teaches a fresh adapter that sees only the bare prompt, under drift budgets toward $\bar\theta_{t-1}$ and θ_0 (§2.4); the adapter is merged into $\bar\theta_t$, the memory is refreshed under the merged weights, and the harness is carried on. One quantity, the residual of a record (Eq. 2), runs through all of it.

2.3 Harness Evolution

Proposal. The harness reads the task through a bounded window $A_t \subset D_t^{\text{train}}$ of whole records filled to a budget B_A, the bounded context of §1: it must decide what to change and what to keep. The frozen checkpoint first answers A_t without a skill; its wrong answers go to the skill writer, so the skill targets what the model now gets wrong rather than restating the task. The base θ_0 then writes the skill in one greedy call from the window and the mistakes. Keeping the writer at θ_0 keeps it fixed across the recursion; co-evolution with the weights is left to future work (Appendix E).

Guard. The residual $r(x, y^*; \theta, K)$ of a record under a model θ executing a registry K is the mean negative log-likelihood of its verified answer with the skill, but not the answer, in context,

$$r = -\frac{1}{|y^*|}\sum_{k=1}^{|y^*|} \log p(y_k^* \mid K(x), x, y_{<k}^*; \theta). \tag{2}$$

A candidate registry is committed only if the mean residual of A_t under $\bar\theta_{t-1}$ is lower than under K_{t-1}; otherwise $K_t = K_{t-1}$: a skill survives only if it helps the checkpoint predict the task.

Residual refresh. After distillation and merge, the residual of every record in $\mathcal{R}_{t-1} \cup A_t$ is recomputed under $\bar\theta_t$ executing K_t, and $\mathcal{R}_t$ keeps whole records in decreasing residual until B_R is filled. What the skill and the new weights together still cannot predict stays in plain text; with one global budget, a record the weights have just absorbed yields its place at once. $\mathcal{R}$ is never replayed and never a training target. It supplies the probes of the next stage's checkpoint-drift constraint: on the records the new checkpoint finds hardest, the student may not leave it. The memory is what the harness keeps; the skill is what it changes.

2.4 Drift-Constrained Distillation

Teacher and student. The teacher p^H is the frozen $\bar\theta_{t-1}$ under a privileged context: the routed skill $K_t(x)$ as the system message and the verified answer y^* replayed as the model's own previous

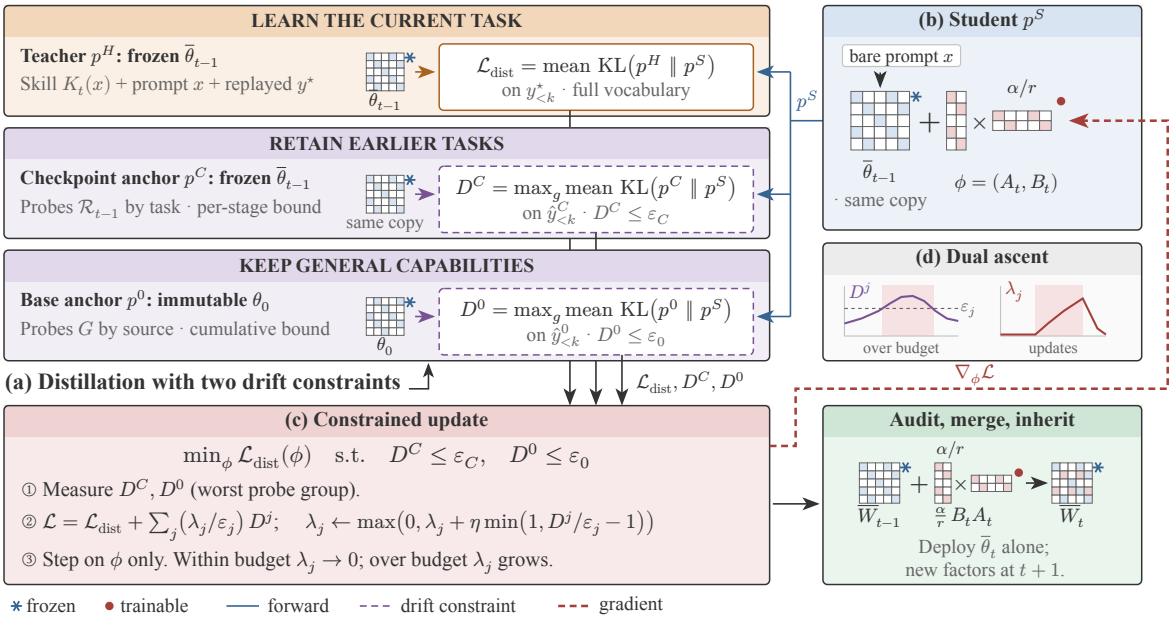


Figure 2: Drift-constrained distillation. (a) One frozen copy of $\bar{\theta}_{t-1}$ serves as both the teacher p^H and the checkpoint anchor p^C; the immutable θ_0 is the general anchor p^0. (b) The student adds fresh factors ϕ to the same weights and sees only the bare prompt. (c) Each update measures both drifts, steps on ϕ, and moves the multipliers by dual ascent. (d) A multiplier grows only while its budget is exceeded.

turn. The student p^S is the same weights with a freshly initialized LoRA adapter (Hu et al., 2021) and sees only the bare prompt (Figure 2a,b). The context carries no wording about the answer, so the student learns the answer and not the scaffolding around it.

With $p^H_k = p^H(\cdot \mid K_t(x), x, y^*, y^*_{<k})$ and $p^S_k = p^S(\cdot \mid x, y^*_{<k})$, the student minimizes the full-vocabulary forward KL divergence at every position of the verified answer,

$$\mathcal{L}_{\text{dist}} = \mathbb{E}_{(x,y^*)} \frac{1}{|y^*|} \sum_{k=1}^{|y^*|} \text{KL}\big(p^H_k \,\|\, p^S_k\big). \quad (3)$$

Scoring the verified answer's own prefix rather than student samples closes degenerate paths such as echoing the prompt, on which teacher and student agree and the gradient vanishes; the price is that the objective is not on-policy.

Drift. A probe is a prompt x with the first n greedy tokens $\hat{y}$ of an anchor a; with $p^a_k = p^a(\cdot \mid x, \hat{y}_{<k})$ and p^S_k likewise, drift over probe groups P_g is the worst group's mean divergence,

$$D(P; a) = \max_g \mathbb{E}_{P_g} \frac{1}{n} \sum_{k=1}^{n} \text{KL}(p^a_k \,\|\, p^S_k). \quad (4)$$

Only the opening is scored because it decides the mode of a response: the failures in Figure 1, a wrong option and a dropped choice letter, are flips of the first few tokens that a long average would hide. The worst group is taken because a collapse confined to one task, Seq-LoRA's correct ScienceQA answers falling from 94 to 15 across the stages of the same figure, barely moves a mean over all probes, which is also why we report $\Delta C_{\min}$.

Two anchors. Checkpoint drift D^C anchors $\bar{\theta}_{t-1}$ on probes from $\mathcal{R}_{t-1}$, grouped by task, and bounds per-stage drift and the recently learned tasks; base drift D^0 anchors θ_0 on G, grouped by source, and bounds cumulative drift and general capability. Each bounds what the other cannot: the previous checkpoint keeps every step small, but small steps accumulate; the base cannot protect a task it never knew. The budgets ε_C and ε_0 are in units of divergence, so one pair serves every stage, which a penalty coefficient could not since the scale of the loss changes from stage to stage.

Enforcement. The stage solves

$$\min_\phi \ \mathcal{L}_{\text{dist}}(\phi) \ \text{ s.t. } \ D^C \le \varepsilon_C, \ D^0 \le \varepsilon_0, \quad (5)$$

over the adapter parameters ϕ with multipliers $\lambda_C, \lambda_0 \ge 0$: each update measures both drifts, adds every active constraint's worst-group divergence to the loss with weight λ_j/ε_j, steps on ϕ only, and moves the multipliers by bounded dual

ascent,

$$\lambda_j \leftarrow \max(0,\, \lambda_j + \eta \min(1,\, D^j/\varepsilon_j - 1)), \quad (6)$$

so that within budget a multiplier decays to zero and leaves learning alone, and over budget it grows and pulls the student back. The final adapter's drift is audited against the original budgets (Figure 2c).

One checkpoint, three roles. One copy of the previous checkpoint is resident as the student's backbone, the teacher p^H, its forward pass with the adapter disabled, and the anchor p^C; both anchors' states on their fixed probes are cached. With a fixed number of updates and probes per stage, the cost does not grow with t; only $\bar{\theta}_t$ is deployed, and budgets and constants are listed in Appendix A.

3 Experiments

Our experiments evaluate HERD under a unified protocol to answer three questions. **Q1.** Does HERD outperform purely parametric and purely non-parametric methods? **Q2.** As the number of CL stages grows, does HERD maintain long-horizon stability? **Q3.** Are harness evolution, high-residual memory, and the two drift anchors all indispensable?

3.1 Experimental Settings

Benchmark. TRACE+17 appends 17 Super-NaturalInstructions tasks (Wang et al., 2022) to the eight TRACE tasks (Wang et al., 2023b), because eight stages are too few to tell stability from slow collapse. Each of the 25 tasks has 500 training, 100 validation, and 100 test examples; no prompt appears in more than one task or split. A G-test of five categories, 200 held-out items each, tracks general capability: knowledge, mathematics, commonsense, instruction following, and multilingual understanding (MMLU, GSM8K, HellaSwag, IFEval, XCOPA; Hendrycks et al., 2020; Cobbe et al., 2021; Zellers et al., 2019; Zhou et al., 2023; Ponti et al., 2020). All methods start from Qwen3.5-4B with the same decoding, length limits, and scorers; all weight updates share one budget, 32 updates of 32 records per stage. After every stage we score the current task; after stages 4, 8, 16, and 25 we also score all seen tasks and the G-test (Appendix B).

Baselines. *Non-parametric*: ACE (Zhang et al., 2026c) grows a playbook; HCL (Kang et al., 2026) keeps a harness edit only if it improves the current task and breaks no retained example; both attach the harness to every prompt at inference. *Parametric*: Seq-LoRA trains one adapter through the stream, the floor; uniform replay keeps a memory the size of HERD's residual memory, filled by uniform sampling; OPR (Chen et al., 2026) replays the model's own best earlier answers; TreeLoRA (Qian et al., 2025) and MoRAM (Lu et al., 2026) isolate updates, in a gradient-similarity tree and in rank-1 experts; SDFT (Shenfeld et al., 2026) self-distills from a demonstration-conditioned teacher, without a harness. *Hybrid*: OPHSD (Zhao et al., 2026) distills a fixed harness into the weights and deploys without it; the harness never changes and drift is not budgeted. *Controls*: the base model as is, and the same model under a fixed harness that never learns. Baselines run the official code where licensed (Appendix B).

Metrics. Acquisition, the mean of $R_{j,j}$, is each task's score right after it is learned; AA is what remains after stage 25; F is what is lost in between (Eq. 1). We report both the mean change over the five G-test categories, ΔC, and the worst, $\Delta C_{\min}$, because a collapse in one category can hide behind a mean of five (§2.4). All numbers are means over three seeds, with standard deviations and 95% bootstrap intervals over items.

3.2 Main Results

Table 1 reports every method after stage 25 and Figure 3 the trajectories behind it. HERD lies on the frontier of the three families: it acquires as much as the strongest parametric method, retains more than any method that updates weights, loses the least general capability among them, and deploys without context.

Non-parametric methods. ACE and HCL finish within 1.4 points of the unchanged base model (AA 0.427 and 0.433 against 0.419): the stream leaves almost no trace in their harnesses. ACE's playbook reaches 80 bullets and 15,410 tokens by task 2 and stops growing; once it fills the prompt, nearly every later training call exceeds the 16,384-token input ceiling and is rolled back, yet every test question still carries those 15,410 tokens of context. HCL commits none of its proposals in 25 stages, so its scores amount to the base model under a fixed system prompt. HERD, scored bare, is 27 points above both; the paired 95% interval against HCL-Stable excludes zero (Table 2).

Method	Family	AA↑	Acq.↑	F↓	BWT↑	G-test↑	ΔC ↑	$\Delta C_{\min}$ ↑
HERD	This work	**0.706**	0.713	0.010	−0.007	0.774	−0.063	−0.130
Base	Control	0.419	0.419	0.000	0.000	0.837	0.000	0.000
Static harness	Control	0.432	0.432	0.000	0.000	0.791	−0.046	−0.135
HCL-Stable	Context	0.433	0.435	0.003	−0.002	**0.823**	**−0.014**	**−0.085**
ACE	Context	0.427	0.428	**0.001**	**−0.001**	0.748	−0.089	−0.320
Seq-LoRA	Weights	0.694	**0.715**	0.025	−0.022	0.597	−0.240	−0.665
Uniform replay	Weights	0.699	0.711	0.020	−0.013	0.599	−0.238	−0.675
OPR	Weights	0.690	0.705	0.024	−0.016	0.614	−0.223	−0.625
TreeLoRA	Weights	0.686	0.711	0.032	−0.026	0.605	−0.232	−0.635
MoRAM	Weights	0.693	0.701	0.019	−0.009	0.637	−0.200	−0.650
SDFT	Weights	0.021	0.024	0.003	−0.003	0.172	−0.665	−0.770
OPHSD	Hybrid	0.620	0.639	0.021	−0.020	0.582	−0.255	−0.645

Table 1: TRACE+17 results after stage 25 under a matched model, stream, splits, prompts, and schedule; means over three seeds. Bold: best learning method (controls excluded) per column. BWT: mean$_{j<T}(R_{T,j} - R_{j,j})$.

Parametric methods. HERD's acquisition, 0.713, is statistically indistinguishable from Seq-LoRA's 0.715, the plasticity floor (paired difference -0.002, interval $[-0.010, 0.007]$), and from uniform replay and TreeLoRA; it is 0.8 points above OPR and 7.4 above OPHSD. Its AA of 0.706 is the highest in the table, significantly above Seq-LoRA, OPR, TreeLoRA, and OPHSD (1.2 to 8.7 points) and tied with uniform replay (0.7 points, interval $[-0.001, 0.016]$). Forgetting is 0.010, under half of Seq-LoRA's, OPR's, TreeLoRA's, and OPHSD's and about half of uniform replay's and MoRAM's, with paired intervals excluding zero against Seq-LoRA, uniform replay, OPR, TreeLoRA, and OPHSD (except SDFT's 0.003 accompanies an AA of 0.021). SDFT's F of 0.003 reflects collapse, not retention: it acquires only 0.024 (Appendix D). Uniform replay holds exactly as many tokens as HERD's residual memory and still forgets like the others (F 0.020, ΔC -0.238): the gap is not in how much is stored, and §3.3 locates it in what is kept and what is constrained.

General capability. The parametric baselines other than SDFT pay for new tasks with old abilities: ΔC is -0.20 to -0.24, and the worst category falls by 62 to 68 points. The loss concentrates in GSM8K and IFEval, which collapse within four stages (Seq-LoRA: from 0.89 and 0.82 to 0.18 and 0.33) and never recover, while knowledge, commonsense, and multilingual scores stay within a few points of the base (Figure 3c). HERD's ΔC of -0.063 and $\Delta C_{\min}$ of -0.130 are the smallest of the weight-updating methods by a wide margin: every parametric baseline trails its G-test macro average by 13.7 to 60.2 points, and the four parametric baselines with paired bootstrap comparisons in Table 2 by 16.0 to 17.7 points; its largest loss, 13 points, falls in the same instruction-following category.

Cost. A HERD stage runs a fixed number of updates and probes, so its training time tracks answer length, not the stage index: 316 s at stage 1, 284 s at stage 25, 10,723 s in total against Seq-LoRA's 10,463 s. OPHSD, which regenerates its teacher context by rollouts at every stage, needs 659,000 s; ACE pays its 15,410 tokens at every inference.

Long-horizon stability. On the first eight tasks (Figure 3a), HERD's mean stays within half a point from stage 8 to stage 25, 0.664 at both ends. Every parametric baseline moves more: Seq-LoRA falls to 0.372 at stage 12 before recovering to 0.637, TreeLoRA to 0.593, and OPR and MoRAM decline to 0.621 and 0.625. On the G-test, the parametric baselines take most of their loss by stage 4 or 8 and then plateau, while HERD loses 1.5 points by stage 4 and 6.3 by stage 25. The two recursive self-distillation baselines instead drift further at every checkpoint. OPHSD's G-test falls through 0.741, 0.661, 0.613, and 0.582 at stages 4, 8, 16, and 25, and its first eight tasks from 0.524 to 0.506. SDFT, whose teacher is asked to include its thinking process, produces answers that lengthen stage after stage, from 345 tokens on average at stage 1 to the 5,120-token ceiling from stage 17 on; 14,098 of its 17,200 test predictions are cut off, and it ends at AA 0.021 and G-test 0.172. HERD also distills the model from itself under a privileged context, but its teacher context is a skill and the verified answer rather than text generated by the drifting model,

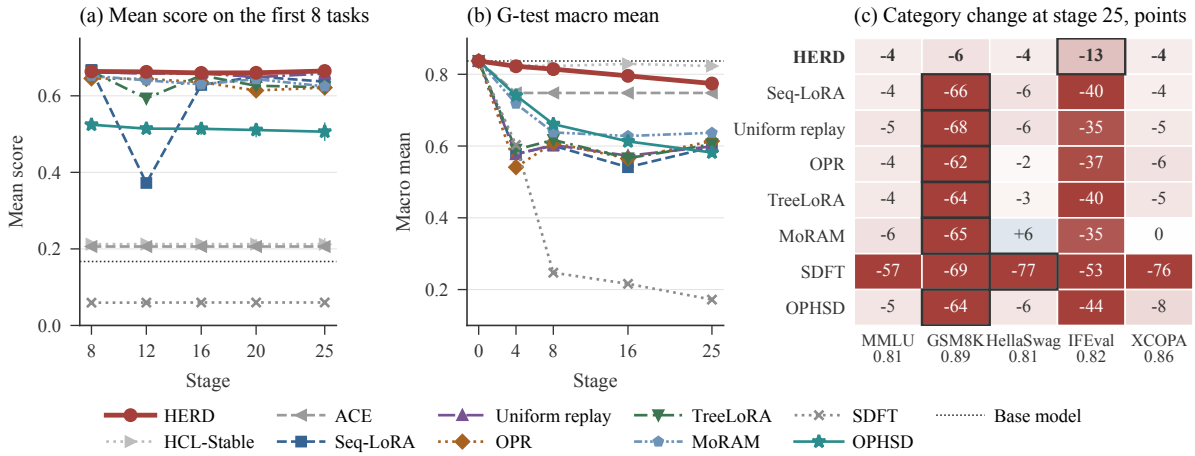


Figure 3: Long-horizon stability on TRACE+17. (a) Mean score on tasks 1 to 8 at the retention stages. (b) G-test macro mean at the anchor stages; error bars are one standard deviation over three seeds, the dotted line is the base model. (c) Change in each G-test category after stage 25 against the base model, in points, for the weight-updating methods; the outlined cell is each method's $\Delta C_{\min}$. Per-stage trajectories and HERD's drift audit are in Figure 7.

its loss is scored on the verified answer, and two anchors bound drift (§2.4): the audited stage-25 drifts are 0.021 and 0.041 against budgets of 0.05.

3.3 Ablation Study

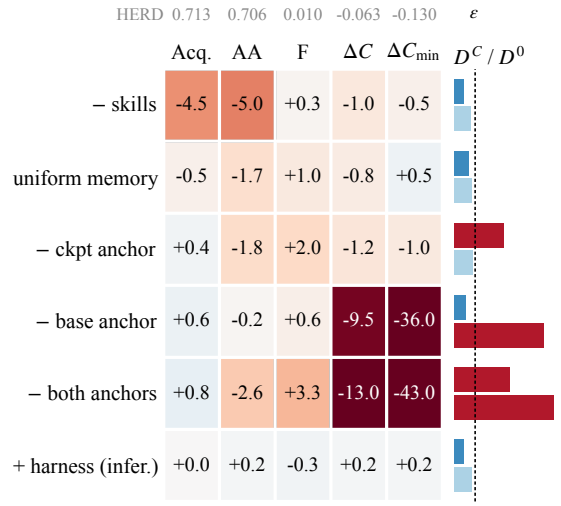


Figure 4: Ablations after stage 25: changes from full HERD in points (red is worse; colors clipped at ± 10). The strip shows audited drifts D^C (upper) and D^0 (lower) against $\varepsilon = 0.05$. Single seed; full table in Appendix C.

Figure 4 removes or replaces one component of HERD at a time and reports the change after stage 25; Table 3 adds the SFT objective and the ε_C sweep. Each component protects one quantity.

Harness evolution. Without skills, the teacher sees only the verified answer and nothing is proposed or committed; memory, probes, and anchors are unchanged. Acquisition falls by 4.5 points and AA by 5.0, the largest losses in both columns, while forgetting and general capability move by at most a point. Without both anchors, the harness evolves and nothing constrains the weights: acquisition holds, and retention and general capability are those of a parametric baseline. OPHSD, whose harness never changes and whose drift is never budgeted, has lower acquisition and a larger general-capability loss than HERD (Acq. 0.639, $\Delta C_{\min}$ -0.645). Only the full model, which evolves the harness under constraint, is on the frontier.

High-residual memory. Filling the memory by uniform sampling under the same budget leaves acquisition within half a point, but forgetting doubles, from 0.010 to 0.020, the value of uniform replay in Table 1. The checkpoint probes are drawn from the memory, so uniform selection anchors the checkpoint on records the weights already predict and leaves the hardest earlier records unguarded; the stage-end D^C stays within budget at 0.034.

Two anchors. Without the checkpoint anchor, forgetting rises to 0.030, close to the worst parametric baseline's 0.032, and the audited D^C ends at 0.118, more than twice the budget. Without the base anchor, the checkpoint constraint remains active, but base drift accumulates: the stage-end D^0 grows from 0.104 at stage 4 to 0.212 at stage 25, $\Delta C_{\min}$ falls from -0.031 to -0.490, and mathematics drops from 0.885 to 0.395 where the full model keeps 0.823. With neither anchor, forgetting reaches 0.044, above every parametric baseline, and $\Delta C_{\min}$ reaches -0.560, close to their -0.63

to -0.68.

Objective and budget. Replacing distillation by cross-entropy on the verified answers, under the same adapter, budget, and constraints, doubles the general loss, to ΔC -0.123 and $\Delta C_{\min}$ -0.320, with both audited drifts within budget; this suggests that the teacher distribution limits drift where the probes do not reach. The budget ε_C trades forgetting against acquisition in the expected direction (Appendix C). Attaching the stage-25 harness at inference reduces forgetting by 0.3 points and changes the other metrics by at most 0.2 points (last row).

4 Related Work

Parametric continual learning. Surveys group LLM continual learning by training stage (Wu et al., 2024; Shi et al., 2025); we group it by where experience is kept. Structural methods isolate task updates in low-rank subspaces, adapter trees, per-task experts, or retained prompts and modules (Wang et al., 2023a; Yang et al., 2026; Qian et al., 2025; Lu et al., 2026; Razdaibiedina et al., 2023; Zhao et al., 2024). Replay methods synthesize or select the data mixed into training (Huang et al., 2024; Chen et al., 2026; Feng et al., 2026; Lee et al., 2026). Self-supervised methods write their own fine-tuning data (Zweiger et al., 2025), distill from a demonstration-conditioned self (Shenfeld et al., 2026), or consolidate fast memories into stable weights (Behrouz et al., 2026). Other work finds part of measured forgetting spurious (Zheng et al., 2025), consolidates in weight space (Cho et al., 2026), or reports limits of on-policy self-distillation (Wang et al., 2026a). HERD also updates low-rank factors by self-distillation, but a harness decides what to internalize and drift budgets bound each step.

Non-parametric continual learning. Non-parametric methods adapt an external memory, context, or harness around a fixed model, where the stability–plasticity tension reappears through retrieval (Gutiérrez et al., 2025; Hu et al., 2026). Recent systems consolidate memory banks (Ye et al., 2026a), evolve contexts into playbooks (Zhang et al., 2026c), gate harness edits on improvement and retention checks (Kang et al., 2026), or refine harnesses over task streams (Wei et al., 2026; Liu et al., 2026). All pass experience through a bounded context, so nothing accumulates in the model itself (§1). HERD keeps such a harness as its fast layer but internalizes it into the weights after every stage.

Hybrid continual learning. A few methods combine the two. OEL consolidates deployment experience online (Ye et al., 2026b), Continual Harness updates the model from teacher-relabeled harness rollouts (Karten et al., 2026), and EvoSC consolidates trajectories into soft prompts (Yu et al., 2026). HERD keeps only skills and the high-residual samples they miss, and deploys the weights alone.

Distillation and harness adaptation. Context distillation internalizes instructions and examples into the weights (Snell et al., 2022). On-policy variants condition the teacher on experiential knowledge (Ye et al., 2026c) or on a harness (Zhao et al., 2026), and RL's Razor ties forgetting to the distributional shift that on-policy training keeps small (Shenfeld et al., 2025). HERD's privileged teacher is of this kind, but it keeps the harness across stages where OPHSD discards it, bounds drift explicitly, and scores verified answer prefixes instead of student rollouts. Self-Harness accepts edits only after regression testing on fixed benchmarks (Zhang et al., 2026a); HERD's commit gate is a simpler version of the same check (§2.3). Two evaluations separate updating a harness from benefiting from it (Lin et al., 2026) and ask whether optimizer gains compound over a continual stream (Wang et al., 2026b).

5 Conclusion

We presented HERD, a hybrid continual learning system that learns in the harness and internalizes it into the weights. The harness evolves with the stream, iterating on its skills to abstract reusable patterns and keeping the high-residual samples they do not cover in a bounded memory. Distillation then internalizes the harness under drift constraints with two anchors, the previous checkpoint for per-stage drift and the base model for cumulative drift; only the weights are deployed. On the 25-stage TRACE+17 stream, HERD matches the strongest parametric baseline in acquisition, forgets the least among weight-updating methods that avoid collapse, and loses the least general capability. Removing any part costs acquisition, retention, or general capability, and attaching the final harness at inference barely changes the scores.

Limitations

Our empirical validation is currently scoped to one base model and one 25-stage stream rather than a large-scale sweep. HERD also relies on a verified answer y^* for every training record and assumes a stream with known task boundaries, one task per stage, and no revisits. A detailed discussion regarding these boundaries and the corresponding plans is provided in Appendix E.

References

Zeyuan Allen-Zhu and Yuanzhi Li. 2025. Physics of language models: Part 3.3, knowledge capacity scaling laws. In *The Thirteenth International Conference on Learning Representations*.

Fernando Alva-Manchego, Louis Martin, Antoine Bordes, Carolina Scarton, Benoît Sagot, and Lucia Specia. 2020. ASSET: A dataset for tuning and evaluation of sentence simplification models with multiple rewriting transformations. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4668–4679, Online. Association for Computational Linguistics.

Ali Behrouz, Farnoosh Hashemi, Adel Javanmard, and Vahab Mirrokni. 2026. Language models need sleep: Learning to self-modify and consolidate memories. *arXiv preprint arXiv:2606.03979*.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901.

Wenhu Chen, Hanwen Zha, Zhiyu Chen, Wenhan Xiong, Hong Wang, and William Yang Wang. 2020. HybridQA: A dataset of multi-hop question answering over tabular and textual data. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1026–1036, Online. Association for Computational Linguistics.

Yan Chen, Taojie Zhu, Meng Zhang, Xin Chen, Jiaqi Huang, Dongyang Xu, and Yizhi Wang. 2026. On-policy replay for continual supervised fine-tuning. *arXiv preprint arXiv:2605.29495*.

Dongkyu Cho, Taesup Moon, Rumi Chunara, Kyunghyun Cho, and Sungmin Cha. 2026. Forget forgetting: Continual learning in a world of abundant memory. In *The Fourteenth International Conference on Learning Representations*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

A. Feder Cooper, Mark A. Lemley, Allison Casasola, Ahmed Ahmed, Aaron Gokaslan, Amy B. Cyphert, Christopher De Sa, Daniel E. Ho, and Percy Liang. 2026. Extracting memorized pieces of (copyrighted) books from open-weight language models. In *Third Conference on Language Modeling*.

DeepSeek-AI. 2026. DeepSeek Harness: Everything is a plugin. Open-source software, developer preview.

Grégoire Delétang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, Christopher Mattern, Jordi Grau-Moya, Li Kevin Wenliang, Matthew Aitchison, Laurent Orseau, Marcus Hutter, and Joel Veness. 2024. Language modeling is compression. In *The Twelfth International Conference on Learning Representations*.

Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Rupam Mahmood, and Richard S. Sutton. 2024. Loss of plasticity in deep continual learning. *Nature*, 632:768–774.

Yujie Feng, Hao Wang, Jian Li, Xu Chu, Zhaolu Kang, Yiran Liu, Yasha Wang, Philip S. Yu, and Xiao-Ming Wu. 2026. FOREVER: Forgetting curve-inspired memory replay for language model continual learning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24945–24965. Association for Computational Linguistics.

Claire Gardent, Anastasia Shimorina, Shashi Narayan, and Laura Perez-Beltrachini. 2017. The WebNLG challenge: Generating text from RDF data. In *Proceedings of the 10th International Conference on Natural Language Generation*, pages 124–133, Santiago de Compostela, Spain. Association for Computational Linguistics.

Alec Go, Richa Bhayani, and Lei Huang. 2009. Twitter sentiment classification using distant supervision. Technical report, Stanford University.

Harsha Gurulingappa, Abdul Mateen Rajput, Angus Roberts, Juliane Fluck, Martin Hofmann-Apitius, and Luca Toldo. 2012. Development of a benchmark corpus to support the automatic extraction of drug-related adverse effects from medical case reports. *Journal of Biomedical Informatics*, 45(5):885–892.

Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. From RAG to memory: Non-parametric continual learning for large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 21497–21515.

Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. 2023. MLX: Efficient and flexible machine learning on Apple silicon.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Qisheng Hu, Quanyu Long, and Wenya Wang. 2026. When continual learning moves to memory: A study of experience reuse in LLM agents. *arXiv preprint arXiv:2604.27003*.

Yebowen Hu, Timothy Ganter, Hanieh Deilamsalehy, Franck Dernoncourt, Hassan Foroosh, and Fei Liu. 2023. MeetingBank: A benchmark dataset for meeting summarization. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16409–16423, Toronto, Canada. Association for Computational Linguistics.

Jianheng Huang, Leyang Cui, Ante Wang, Chengyi Yang, Xinting Liao, Linfeng Song, Junfeng Yao, and Jinsong Su. 2024. Mitigating catastrophic forgetting in large language models with self-synthesized rehearsal. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1416–1428. Association for Computational Linguistics.

Lifu Huang, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2019. Cosmos QA: Machine reading comprehension with contextual commonsense reasoning. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2391–2401, Hong Kong, China. Association for Computational Linguistics.

Marcus Hutter. 2005. *Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability*. Springer, Berlin.

Borui Kang, Jinrui Gu, Junhan Lv, Wenbin Li, Lei Wang, and Yang Gao. 2026. Harness continual learning: Continual adaptation beyond model parameters. *arXiv preprint arXiv:2608.19013*.

Seth Karten, Joel Zhang, Tersoo Upaa, Ruirong Feng, Wenzhe Li, Chengshuai Shi, Chi Jin, and Kiran Vodrahalli. 2026. Continual harness: Online adaptation for self-improving foundation agents. *arXiv preprint arXiv:2605.09998*.

Byeongchang Kim, Hyunwoo Kim, and Gunhee Kim. 2019. Abstractive summarization of Reddit posts with multi-level memory networks. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2519–2531, Minneapolis, Minnesota. Association for Computational Linguistics.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

Minjae Lee, Minhyuk Seo, Tingyu Qu, Tinne Tuytelaars, and Jonghyun Choi. 2026. OASIS: Online sample selection for continual instruction tuning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3491–3515. Association for Computational Linguistics.

Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain. Association for Computational Linguistics.

Minhua Lin, Juncheng Wu, Zijun Wang, Zhan Shi, Yisi Sang, Bing He, Zewen Liu, Tianxin Wei, Zongyu Wu, Zhiwei Zhang, Dakuo Wang, Xiang Zhang, Benoit Dumoulin, Cihang Xie, Yuyin Zhou, Suhang Wang, and Hanqing Lu. 2026. Harness updating is not harness benefit: Disentangling evolution capabilities in self-evolving LLM agents. *arXiv preprint arXiv:2605.30621*.

Zewen Liu, Zhan Shi, Yisi Sang, Bing He, Minhua Lin, Tianxin Wei, Dakuo Wang, Benoit Dumoulin, Wei Jin, and Hanqing Lu. 2026. Adaptive auto-harness: Sustained self-improvement for agentic system deployment on open-ended task streams. *arXiv preprint arXiv:2606.01770*.

Haodong Lu, Chongyang Zhao, Minhui Xue, Lina Yao, Kristen Moore, and Dong Gong. 2026. Little by little: Continual learning via incremental mixture of rank-1 associative memory experts. In *Proceedings of the 43rd International Conference on Machine Learning*.

Pan Lu, Swaroop Mishra, Tanglin Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. 2022. Learn to explain: Multimodal reasoning via thought chains for science question answering. In *Advances in Neural Information Processing Systems*, volume 35, pages 2507–2521. Curran Associates, Inc.

Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, and 3 others. 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation. *CoRR*, abs/2102.04664.

Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, Benjamin Bossan, and Marian Tietz. 2022. PEFT: State-of-the-art parameter-efficient fine-tuning methods.

Mantas Mazeika, Alice Gatti, Cristina Menghini, Udari Madhushani Sehwag, Shivam Singhal, Yury Orlovskiy, Steven Basart, Manasi Sharma, Denis Peskoff, Elaine Lau, Jaehyuk Lim, Lachlan Carroll, Alice Blair, Vinaya Sivakumar, Sumana Basu, Brad Kenstler, Yuntao Ma, Julian Michael, Xiaoke Li, and 28 others. 2025. Remote labor index: Measuring AI automation of remote work. *arXiv preprint arXiv:2510.26787*.

James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. 1995. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457.

Michael McCloskey and Neal J. Cohen. 1989. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press.

Swaroop Mishra, Arindam Mitra, Neeraj Varshney, Bhavdeep Sachdeva, Peter Clark, Chitta Baral, and Ashwin Kalyan. 2022. NumGLUE: A suite of fundamental yet challenging mathematical reasoning tasks. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3505–3523, Dublin, Ireland. Association for Computational Linguistics.

John Xavier Morris, Chawin Sitawarin, Chuan Guo, Narine Kokhlikyan, G. Edward Suh, Alexander M Rush, Kamalika Chaudhuri, and Saeed Mahloujifar. 2026. How much can language models memorize? In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 90392–90411. PMLR.

Nasrin Mostafazadeh, Nathanael Chambers, Xiaodong He, Devi Parikh, Dhruv Batra, Lucy Vanderwende, Pushmeet Kohli, and James Allen. 2016. A corpus and cloze evaluation for deeper understanding of commonsense stories. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 839–849, San Diego, California. Association for Computational Linguistics.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, and 2 others. 2019. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc.

Edoardo Maria Ponti, Goran Glavaš, Olga Majewska, Qianchu Liu, Ivan Vulić, and Anna Korhonen. 2020. XCOPA: A multilingual dataset for causal commonsense reasoning. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2362–2376, Online. Association for Computational Linguistics.

Ye Qi, Devendra Sachan, Matthieu Felix, Sarguna Padmanabhan, and Graham Neubig. 2018. When and why are pre-trained word embeddings useful for neural machine translation? In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 529–535, New Orleans, Louisiana. Association for Computational Linguistics.

Yu-Yang Qian, Yuan-Ze Xu, Zhen-Yu Zhang, Peng Zhao, and Zhi-Hua Zhou. 2025. TreeLoRA: Efficient continual learning via layer-wise LoRAs guided by a hierarchical gradient-similarity tree. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 50066–50085. PMLR.

Qwen Team. 2026. Qwen3.5: Towards native multimodal agents.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas. Association for Computational Linguistics.

Veselin Raychev, Pavol Bielik, and Martin Vechev. 2016. Probabilistic model for code with decision trees. *ACM SIGPLAN Notices*, 51(10):731–747.

Anastasia Razdaibiedina, Yuning Mao, Rui Hou, Madian Khabsa, Mike Lewis, and Amjad Almahairi. 2023. Progressive prompts: Continual learning for language models. In *The Eleventh International Conference on Learning Representations*.

Annette Rios, Nicolas Spring, Tannon Kew, Marek Kostrzewa, Andreas Säuberli, Mathias Müller, and Sarah Ebling. 2021. A new dataset and efficient baselines for document-level text simplification in German. In *Proceedings of the Third Workshop on New Frontiers in Summarization*, pages 152–161, Online and in Dominican Republic. Association for Computational Linguistics.

Adam Roberts, Colin Raffel, and Noam Shazeer. 2020. How much knowledge can you pack into the parameters of a language model? In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5418–5426, Online. Association for Computational Linguistics.

Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S. Gordon. 2011. Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In *AAAI Spring Symposium on Logical Formalizations of Commonsense Reasoning*.

Agam Shah, Suvan Paturi, and Sudheer Chava. 2023. Trillion dollar words: A new financial dataset, task & market analysis. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6664–6679, Toronto, Canada. Association for Computational Linguistics.

Idan Shenfeld, Mehul Damani, Jonas Hübotter, and Pulkit Agrawal. 2026. Self-distillation enables continual learning. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 111542–111558. PMLR.

Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. 2025. RL's razor: Why online reinforcement learning forgets less. *arXiv preprint arXiv:2509.04259*.

Haizhou Shi, Zihao Xu, Hengyi Wang, Weiyi Qin, Wenyuan Wang, Yibin Wang, Zifeng Wang, Sayna Ebrahimi, and Hao Wang. 2025. Continual learning of large language models: A comprehensive survey. *ACM Computing Surveys*, 58(5):1–42.

Charlie Snell, Dan Klein, and Ruiqi Zhong. 2022. Learning by distilling context. *arXiv preprint arXiv:2209.15189*.

Bertie Vidgen, Dong Nguyen, Helen Margetts, Patricia Rossini, and Rebekah Tromble. 2021. Introducing CAD: the contextual abuse dataset. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2289–2303, Online. Association for Computational Linguistics.

Meng Wang, Haohan Zhao, Wenzhuo Liu, Lu Yang, Geng Liu, Haiyang Guo, Guo-Sen Xie, Gaofeng Meng, Hongbin Liu, and Fei Zhu. 2026a. Denser $\neq$ better: Limits of on-policy self-distillation for continual post-training. *arXiv preprint arXiv:2607.01763*.

Wenxiao Wang, Priyatham Kattakinda, and Soheil Feizi. 2026b. Do agent optimizers compound? a continual-learning evaluation on Terminal-Bench 2.0. *arXiv preprint arXiv:2607.14004*.

Xiao Wang, Tianze Chen, Qiming Ge, Han Xia, Rong Bao, Rui Zheng, Qi Zhang, Tao Gui, and Xuanjing Huang. 2023a. Orthogonal subspace learning for language model continual learning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 10658–10671. Association for Computational Linguistics.

Xiao Wang, Yuansen Zhang, Tianze Chen, Songyang Gao, Senjie Jin, Xianjun Yang, Zhiheng Xi, Rui Zheng, Yicheng Zou, Tao Gui, Qi Zhang, and Xuanjing Huang. 2023b. TRACE: A comprehensive benchmark for continual learning in large language models. *arXiv preprint arXiv:2310.06762*.

Yizhong Wang, Swaroop Mishra, Pegah Alipoormolabashi, Yeganeh Kordi, Amirreza Mirzaei, Atharva Naik, Arjun Ashok, Arut Selvan Dhanasekaran, Anjana Arunkumar, David Stap, Eshaan Pathak, Giannis Karamanolakis, Haizhi Lai, Ishan Purohit, Ishani Mondal, Jacob Anderson, Kirby Kuznia, Krima Doshi, Kuntal Kumar Pal, and 16 others. 2022. Super-NaturalInstructions: Generalization via declarative instructions on 1600+ NLP tasks. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 5085–5109. Association for Computational Linguistics.

Tianxin Wei, Zhan Shi, Minhua Lin, Bing He, Zewen Liu, Yisi Sang, Yuanchen Bei, Xuying Ning, Jiaru Zou, Ting-Wei Li, Xiao Lin, Yanjun Zhao, Chi Wang, Benoit Dumoulin, Dakuo Wang, Jingrui He, and Hanqing Lu. 2026. Evo-Harness: Context-to-harness skill compilation for self-evolving agents. *arXiv preprint arXiv:2608.15071*.

Wei Wei, Quoc Le, Andrew Dai, and Jia Li. 2018. AirDialogue: An environment for goal-oriented dialogue research. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3844–3854, Brussels, Belgium. Association for Computational Linguistics.

John Wieting and Kevin Gimpel. 2018. ParaNMT-50M: Pushing the limits of paraphrastic sentence embeddings with millions of machine translations. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 451–462, Melbourne, Australia. Association for Computational Linguistics.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.

Tongtong Wu, Linhao Luo, Yuan-Fang Li, Shirui Pan, Thuy-Trang Vu, and Gholamreza Haffari. 2024. Continual learning for large language models: A survey. *arXiv preprint arXiv:2402.01364*.

Wei Xu, Courtney Napoles, Ellie Pavlick, Quanze Chen, and Chris Callison-Burch. 2016. Optimizing statistical machine translation for text simplification.

Transactions of the Association for Computational Linguistics, 4:401–415.

Lina Yang, Yusheng Liao, Yanfeng Wang, and Yu Wang. 2026. SLoRA: Balancing plasticity and forgetting in large language models for continual learning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5437–5454. Association for Computational Linguistics.

Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium. Association for Computational Linguistics.

Chongrui Ye, Yuxiang Liu, Yu Wang, Haofei Yu, Yining Zhao, Ge Liu, Julian McAuley, and Jiaxuan You. 2026a. Auto-Dreamer: Learning offline memory consolidation for language agents. *arXiv preprint arXiv:2605.20616*.

Tianzhu Ye, Li Dong, Qingxiu Dong, Xun Wu, Shaohan Huang, and Furu Wei. 2026b. Online experiential learning for language models. *arXiv preprint arXiv:2603.16856*.

Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. 2026c. On-policy context distillation for language models. *arXiv preprint arXiv:2602.12275*.

Hongzhuo Yu, Fei Zhu, Guo-Sen Xie, and Ling Shao. 2026. Self-consolidation for self-evolving agents. *arXiv preprint arXiv:2602.01966*.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, Florence, Italy. Association for Computational Linguistics.

Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. 2026a. Self-Harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2026b. Darwin gödel machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026c. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*.

Chenye Zhao, Yingjie Li, and Cornelia Caragea. 2023. C-STANCE: A large dataset for Chinese zero-shot stance detection. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13369–13385, Toronto, Canada. Association for Computational Linguistics.

Weixiang Zhao, Shilong Wang, Yulin Hu, Yanyan Zhao, Bing Qin, Xuanyu Zhang, Qing Yang, Dongliang Xu, and Wanxiang Che. 2024. SAPT: A shared attention framework for parameter-efficient continual learning of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11641–11661. Association for Computational Linguistics.

Zhengyang Zhao, Lu Ma, and Wentao Zhang. 2026. Training with harnesses: On-policy harness self-distillation for complex reasoning. In *Advances in Neural Information Processing Systems*, volume 39.

Junhao Zheng, Xidi Cai, Shengjie Qiu, and Qianli Ma. 2025. Spurious forgetting in continual learning of language models. In *The Thirteenth International Conference on Learning Representations*.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*.

Adam Zweiger, Jyothish Pari, Han Guo, Ekin Akyürek, Yoon Kim, and Pulkit Agrawal. 2025. Self-adapting language models. In *Advances in Neural Information Processing Systems*, volume 38.

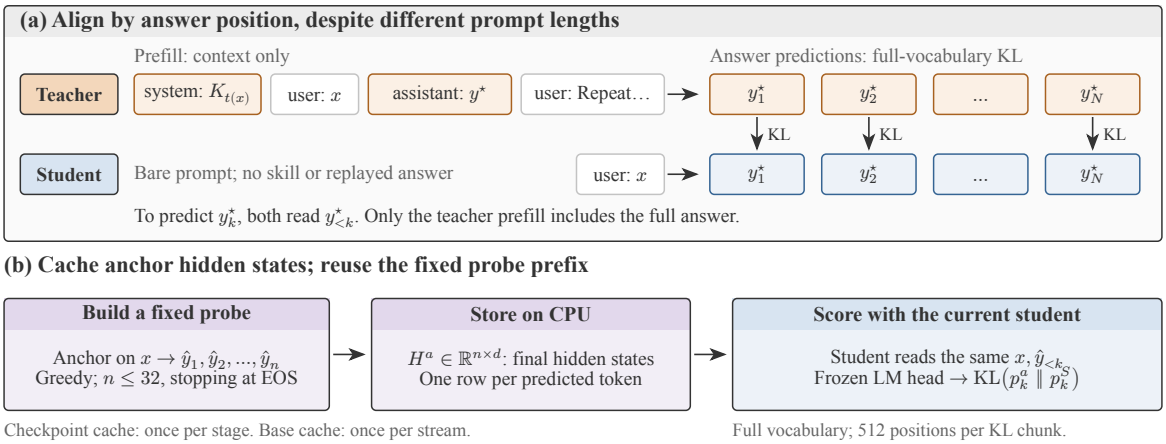


Figure 5: Fixed-prefix scoring. (a) Vertical arrows compare teacher and student distributions at the same answer position by full-vocabulary KL. Both read $y^*_{<k}$; only the teacher's prefill contains the full answer. N includes the closing end-of-turn token; chat-template delimiters are omitted. (b) Anchor hidden states are cached for a fixed greedy probe prefix and projected through the frozen output head when computing drift. Here $n \leq 32$ is the prefix length and d the hidden width.

A Implementation Details

One model, Qwen3.5-4B (Qwen Team, 2026) at a pinned revision, fills every role; all calls go through its chat template with thinking disabled. The weights and chat template are provided by the Qwen Team in the official model repository.[1] Figure 5 details answer-position alignment and probe caching. A skill in the registry K is an identifier, a routing prefix, and instructions. A prompt is routed to the skill with the longest prefix that matches it after NFKC normalization, whitespace collapsing, and lowercasing; the instructions become a fixed system message, and a prompt with no match runs bare. The serialized registry, prefixes included, counts toward $B_K = 24{,}576$ tokens, and one skill holds at most 768. The window has $B_A = 8{,}192$ tokens; the memory has $B_R = 32{,}768$, of which $B_G = 8{,}192$ are reserved for G, so task records occupy at most 24,576.

At stage t, A_t is filled with whole records of D_t^{train} in a seeded order fixed per stage, each counted as prompt plus answer plus a 16-token allowance, until the next record would overflow B_A. The frozen $\bar{\theta}_{t-1}$ answers every record greedily; an answer that differs from y^* after the same normalization is a mistake, and at most four reach the writer. The writer is θ_0 and decodes once, greedily, for at most 768 new tokens. Its prompt lists up to 16 records of A_t, those whose answers open differently first, inputs clipped to 384 tokens and outputs to 128, then the mistakes, and asks for a skill under four headers: Format, Rules, Avoid, Examples. The skill's prefix is the longest common prefix of the task's training prompts; its instructions are the reply from its Format line on, clipped to 768 tokens; it replaces any skill with the same prefix or identifier. The guard is the mean residual of the window under the frozen checkpoint,

$$\bar{r}_t(K) = \frac{1}{|A_t|} \sum_{(x,y^*)\in A_t} r(x, y^*; \bar{\theta}_{t-1}, K),$$
$$K_t = \begin{cases} K' & \text{if } \bar{r}_t(K') < \bar{r}_t(K_{t-1}) \\ & \text{and } \mathrm{size}(K') \leq B_K, \\ K_{t-1} & \text{otherwise,} \end{cases} \tag{7}$$

so a candidate K' is kept only when the skill alone, without a replayed answer in the prefill, makes the window easier to predict.

A residual (Eq. 2) is scored with only the routed skill and bare prompt in the prefill, conditioning on $y^*_{<k}$ at each answer position. The checkpoint probes of stage t come from $\mathcal{R}_{t-1}$: up to eight prompts of at most 2,048 tokens per retained task, each completed by the first 32 greedy tokens of $\bar{\theta}_{t-1}$ on the bare prompt and grouped by task. The general probes are the 48 records of G within B_G, non-test GSM8K, IFEval, and Super-NaturalInstructions records disjoint from the G-test (Appendix B), completed by the first 32 greedy tokens of θ_0 once at the start of the stream and grouped by source. Both probe types stop early at EOS and remain fixed during updates. Each anchor's final hidden states at the probe's prediction positions are cached, once per stage for the check-

[1] https://huggingface.co/Qwen/Qwen3.5-4B

point anchor and once per stream for the base anchor (Figure 5b). The frozen output head projects cached anchor states and current student states to full-vocabulary distributions; KL is computed in chunks of 512 positions.

A training record is one whose bare prompt and verified answer, the answer clipped to 16,384 tokens and closed by the end-of-turn token, fit 16,384 tokens together. The teacher reads the routed skill as the system message, the prompt, the verified answer as its own previous turn, and a final user turn, "Repeat your previous answer exactly, with no other text."; its distribution at each answer position is computed with the adapter disabled. Teacher and student both read the verified-answer prefix $y^*_{<k}$ at position k. Their distributions are aligned by answer-token index rather than absolute sequence position, with the last prompt position predicting the first answer token (Figure 5a). The student adds LoRA factors of rank 32, $\alpha = 64$, and no dropout to every linear layer of the text decoder, leaving out the vision tower, the multi-token-prediction head, and the output head. A stage runs 32 updates of 32 records in a seeded shuffle, with AdamW without weight decay, a learning rate of 10^{-4} decaying linearly to zero, and gradient-norm clipping at 1.0; the final iterate is kept. Each update first measures both drifts on all probes without gradient, then steps on

$$\mathcal{L}(\phi) = \mathcal{L}_{\text{dist}}(\phi) + \sum_{j \in \{C,0\}} [\lambda_j > 0] \, \frac{\lambda_j}{\varepsilon_j} \, \bar{D}^j_{g^*_j}(\phi), \tag{8}$$

where $[\cdot]$ is the indicator, g^*_j is the worst group of drift j at that measurement, and $\bar{D}^j_g$ is that group's mean divergence, the inner mean of Eq. 4; it finally moves each multiplier by Eq. 6 from the same measurement, with $\varepsilon_C = \varepsilon_0 = 0.05$, $\eta = 0.5$, and both multipliers starting at zero. The drifts before the first update and after the last are recorded for every stage; the latter are the audited values of Figure 7 and Table 3.

The adapter is cast to bf16, saved, and merged into the dense weights, which become $\bar{\theta}_t$ and the backbone of the next stage's fresh adapter, so every stage starts from $\Delta W = 0$. The memory is then refreshed under the merged weights. With $\rho_{(1)}, \ldots, \rho_{(n)}$ the records of $\mathcal{R}_{t-1} \cup A_t$, deduplicated by identifier, in decreasing $r(\cdot; \bar{\theta}_t, K_t)$ with ties broken by identifier, $|\rho|$ a record's token length, and τ_i the token count of $\mathcal{R}^{(i)}$, one scan keeps

$$\mathcal{R}^{(i)} = \begin{cases} \mathcal{R}^{(i-1)} \cup \{\rho_{(i)}\} & \text{if } \tau_{i-1} + |\rho_{(i)}| \\ & \quad \leq B_R - B_G, \\ \mathcal{R}^{(i-1)} & \text{otherwise,} \end{cases} \tag{9}$$

from $\mathcal{R}^{(0)} = \emptyset$, and $\mathcal{R}_t = \mathcal{R}^{(n)}$: a record enters only if it fits whole, and a smaller record further down the order may still enter after a larger one is skipped. Within a stage, one copy of $\bar{\theta}_{t-1}$ with its adapter serves as student, teacher, and checkpoint anchor; θ_0 is loaded only to write the skill. Training uses PyTorch 2.6.0 (Paszke et al., 2019), Transformers 5.2.0 (Wolf et al., 2020), and PEFT 0.20.0 (Mangrulkar et al., 2022) in bf16. Each stage records its registry, guard decision, memory, drifts, and the adapter's SHA-256, from which a restarted job replays completed stages.

B Experiment Details

TRACE+17 is built once, with seed 42, from pinned revisions of the TRACE and Super-NaturalInstructions repositories: the eight TRACE tasks in their canonical order, then SuperNI tasks 024, 074, 195, 511, 1219, 574, 192, 177, 064, 346, 108, 111, 1415, 1447, 828, 1728, and 217. Prompts are deduplicated and the splits drawn in a seeded hash order. Each task keeps its official metric: exact match, leading-choice accuracy, ROUGE-L (Lin, 2004), edit similarity, or SARI (Xu et al., 2016). Validation targets and test records are hidden from the model, and no test score selects a candidate or hyperparameter. The G-test draws from pinned snapshots of MMLU, GSM8K, HellaSwag, IFEval, and XCOPA, disjoint from the stream: 200 test items per category, a 40-item probe nested in them, and a separate 40-item development panel. Stage 0 is the base model. After stage t we score the current task; at stages 4, 8, 16, and 25 every seen task and the full G-test; at stages 12 and 20 tasks 1 to 8; at every other stage the 200-item probe. Each cell uses all 100 test items, decoded greedily with thinking off, at most 5,120 new tokens, and 16,384 input tokens; an input over the ceiling counts as an empty prediction. LoRA baselines decode merged checkpoints with vLLM 0.31.0 (Kwon et al., 2023); HERD and MoRAM decode with Transformers; frozen-model methods use MLX (Hannun et al., 2023), PyTorch's MPS backend,[2] or vLLM servers.

[2] https://docs.pytorch.org/docs/stable/notes/mps.html

GSM8K is scored on the final number, IFEval by its strict verifier, and the choice categories by one extractor that reads the last explicit answer.

Every weight-updating method trains for 32 updates of 32 records per stage on sequences of at most 16,384 tokens; longer records are excluded, not truncated. Unless the official code fixes otherwise, the adapter is a rank-32 LoRA with $\alpha = 64$ and no dropout on every linear layer of the text backbone, trained with AdamW at 10^{-4} and gradient clipping at 1.0. Uniform replay keeps a 32,768-token memory, 8,192 of them for 23 reserved general records, filled by uniform sampling without re-entry. OPR-RU samples one response per earlier training prompt at temperature 0.1 and keeps the best-scoring five per stage (replay ratio 0.01), with an equal quota per task. TreeLoRA uses the authors' regularizer weight of 0.5. MoRAM follows the official runner: eight rank-1 atoms per module per stage, $\alpha = 16$, dropout 0.1, top-8 self-activation at temperature 0.03 with an inference gate of 0.2, and Adam$(0.9, 0.95)$ at 5×10^{-4}. SDFT trains a fresh adapter per stage on 32 of its own rollouts per update, at temperature 1 and at most 1,024 new tokens; its teacher is an EMA copy (0.01) of the stage-start weights shown the reference under the official thinking-process template. OPHSD uses the official draft-verify harness on the six stages with at most ten labels and plan-solve elsewhere; the teacher is the stage-start model given the draft labels, or the plan and the reference answer, and the student minimizes the reverse KL on one on-policy rollout per record.

The static harness wraps the frozen model in a fixed system prompt, an empty skill registry, and a Python tool in a 10-second sandbox, for at most 32 turns per item. ACE runs the official offline trainer: batch size 1, one epoch, three reflection rounds, ADD-only curation, the upstream 4,096-token call limit, and a rollback of any sample whose call exceeds the input ceiling. HCL-Stable proposes one alternative per component per stage under a 12,000-token budget and commits it only on a validation gain of at least 0.01 that breaks no item of each earlier task's 20-item validation panel. ACE and MoRAM use the Apache-licensed official code, and OPHSD the official code with the authors' permission; SDFT is a clean-room implementation of the official training step, and HCL-Stable, OPR, and TreeLoRA are implemented from their papers. Learning methods run with seeds 42, 43, and 44 and report the mean and sample standard deviation; intervals are 95% item bootstraps with 4,000 draws, resampled within task and G-test category and paired between HERD and each baseline.

Original dataset sources. In addition to the TRACE benchmark citation, we credit the creators of its constituent datasets: C-STANCE (Zhao et al., 2023), FOMC (Shah et al., 2023), MeetingBank (Hu et al., 2023), Py150 (Raychev et al., 2016) as used in CodeXGLUE (Lu et al., 2021), ScienceQA (Lu et al., 2022), the two NumGLUE tasks (Mishra et al., 2022), and 20Minuten (Rios et al., 2021).

The 17 additional tasks use the instruction-based formulations from Super-NaturalInstructions (Wang et al., 2022); the task identifiers map to the following original resources: 024, Cosmos QA (Huang et al., 2019); 074, SQuAD 1.1 (Rajpurkar et al., 2016); 195, Sentiment140 (Go et al., 2009); 511, Reddit TIFU (Kim et al., 2019); 1219, the multilingual TED corpus, English-to-Spanish translation (Qi et al., 2018); 574, AirDialogue (Wei et al., 2018); 192, HotpotQA (Yang et al., 2018); 177, ParaNMT (Wieting and Gimpel, 2018); 064, the synthetic list-processing task contributed by Pulkit Verma to Super-NaturalInstructions;[3] 346, HybridQA (Chen et al., 2020); 108, the Contextual Abuse Dataset (Vidgen et al., 2021); 111, ASSET (Alva-Manchego et al., 2020); 1415, Emily McMilin's YouTube Caption Corrections;[4] 1447, ADE Corpus V2 (Gurulingappa et al., 2012); 828, COPA (Roemmele et al., 2011); 1728, WebNLG (Gardent et al., 2017); and 217, ROC-Stories (Mostafazadeh et al., 2016). The Super-NaturalInstructions task files identify both the original sources and the contributors of the instruction-based formulations.[5]

C Additional Experimental Results

Acquisition and retention by task. Figure 6 splits Table 1 into its 25 tasks. At the stage a task is learned (panel a), HERD and Seq-LoRA move together, within 3 points on 16 tasks and never more than 7.7 apart (SuperNI 192); HERD is 60 to 92 points above the base on FOMC, Py150, ScienceQA, and the two NumGLUE tasks, on which the base scores at most 0.12. HCL-Stable,

[3] https://github.com/allenai/natural-instructions/blob/master/tasks/task064_all_elements_except_first_i.json

[4] https://github.com/2dot71mily/youtube_captions_corrections

[5] https://github.com/allenai/natural-instructions/tree/master/tasks

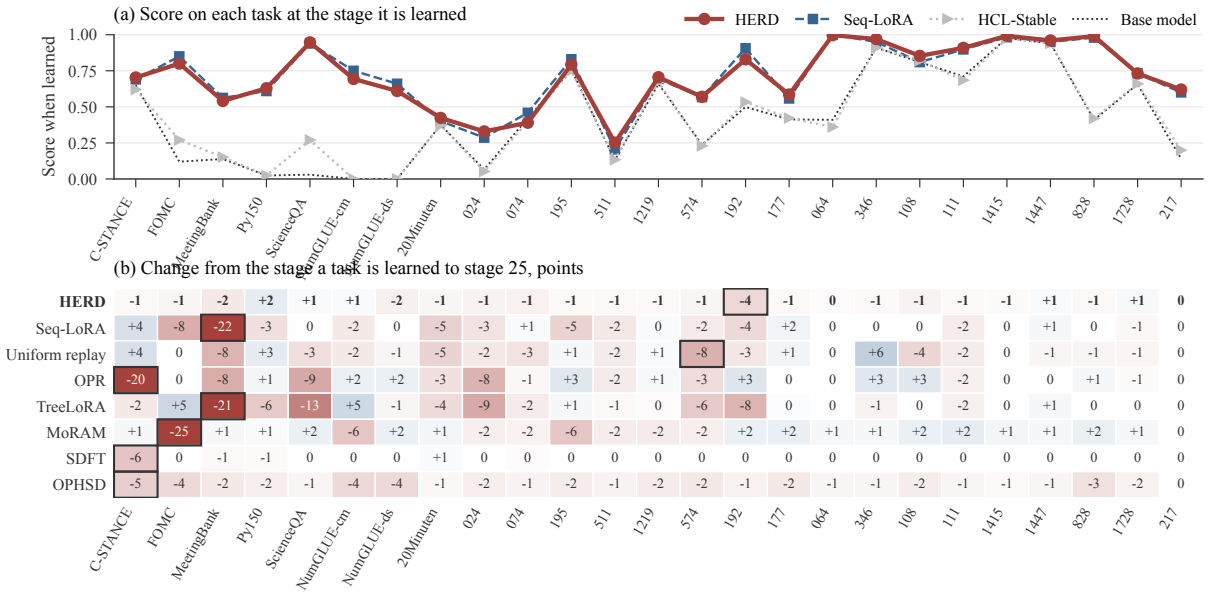


Figure 6: Table 1 by task; means over three seeds. (a) Score on each task at the stage it is learned, for HERD, Seq-LoRA, HCL-Stable, and the base model. (b) Change on each task between the stage it is learned and stage 25, in points, for the weight-updating methods; the outlined cell is each method's largest loss.

which commits nothing, follows the base line. Between that stage and stage 25 (panel b), HERD moves by at most 4 points on any task. Seq-LoRA, TreeLoRA, OPR, and MoRAM each lose 20 points or more on one task, and not the same one: Seq-LoRA and TreeLoRA on MeetingBank (22 and 21), MoRAM on FOMC (25), OPR on C-STANCE (20); uniform replay's largest losses are 8 points, on MeetingBank and SuperNI 574, and OPHSD loses 1 to 5 points on every earlier task. In the parametric baselines, forgetting is concentrated on a few tasks rather than spread over all of them, which is what the worst-group drift of Eq. 4 is defined to catch and a mean over probes would miss.

Paired comparisons. Table 2 lists the differences behind §3.2; each "significantly" there is an interval here that excludes zero. The intervals that include zero are the ties of §3.2: acquisition against Seq-LoRA, uniform replay, and TreeLoRA, and AA against uniform replay. The G-test differences to the parametric baselines in the table, 16.0 to 17.7 points, have intervals no closer than 13.6 points to zero. Against the static harness the G-test difference is -1.7 points with an interval that includes zero: after 25 stages of training, HERD's general capability is statistically indistinguishable from the frozen base under a fixed system prompt.

General capability by category. Figure 7a is the per-category view behind Figure 3c. The parametric baselines lose GSM8K and IFEval between stage 0 and stage 4, Seq-LoRA by 70 and 49 points, and stay there; MMLU, HellaSwag, and XCOPA move by at most 12 points for every method but SDFT, whose five curves fall together. HERD's categories are within 3 points of the base at stage 4 and within 6 at stage 25, except instruction following, which loses 13 points, 9 of them after stage 8. The 200-item probe scored at the other stages shows what the anchors skip: Seq-LoRA's probe mean falls to 0.28, 0.16, and 0.37 at stages 11 to 13 and is back at 0.59 by stage 14, a collapse and recovery that lies between the anchors of Figure 3b; HERD's probe mean never falls below 0.75.

Drift audit. Figure 7b audits HERD's two drifts at every stage. The three-seed mean stage-end values never exceed the budget: D^C reaches at most 0.045 and D^0 at most 0.044. Within a stage, the highest measurement exceeds 0.05 at 9 stages for D^C and 11 for D^0, up to 0.168 and 0.324 at stage 6, and is pulled back before the stage ends; after stage 9 the D^0 peak is at most 0.060. Figure 8e,f put the variants of Table 3 on the same axes. Without the checkpoint anchor, D^C ends above its budget at all 25 stages, between 0.052 and 0.121. Without the base anchor, D^0 ends between 0.055 and 0.235, and the checkpoint constraint that is kept also ends over its budget at 15 stages. At $\varepsilon_C = 0.02$ the stage-end D^C stays between 0.008 and 0.020; at

Baseline	ΔAA	ΔAcq.	ΔF	ΔBWT	ΔG-test
Base	**+0.287 [0.274, 0.299]**	**+0.293 [0.281, 0.306]**	**+0.010 [0.006, 0.016]**	**−0.007 [−0.013, −0.001]**	**−0.063 [−0.088, −0.038]**
Static harness	**+0.274 [0.262, 0.286]**	**+0.281 [0.268, 0.293]**	**+0.010 [0.006, 0.016]**	**−0.007 [−0.013, −0.002]**	−0.017 [−0.044, 0.010]
HCL-Stable	**+0.273 [0.260, 0.286]**	**+0.278 [0.265, 0.291]**	**+0.007 [0.003, 0.013]**	−0.005 [−0.012, 0.001]	**−0.049 [−0.074, −0.024]**
Seq-LoRA	**+0.012 [0.004, 0.021]**	−0.002 [−0.010, 0.007]	**−0.015 [−0.024, −0.009]**	**+0.015 [0.006, 0.023]**	**+0.177 [0.154, 0.200]**
Uniform replay	+0.007 [−0.001, 0.016]	+0.001 [−0.007, 0.010]	**−0.010 [−0.020, −0.004]**	+0.006 [−0.004, 0.017]	**+0.175 [0.150, 0.201]**
OPR	**+0.016 [0.007, 0.025]**	**+0.008 [0.000, 0.015]**	**−0.014 [−0.023, −0.007]**	+0.009 [−0.001, 0.018]	**+0.160 [0.136, 0.184]**
TreeLoRA	**+0.020 [0.011, 0.029]**	+0.002 [−0.006, 0.009]	**−0.021 [−0.030, −0.015]**	**+0.019 [0.010, 0.029]**	**+0.169 [0.144, 0.194]**
OPHSD	**+0.087 [0.073, 0.100]**	**+0.074 [0.061, 0.087]**	**−0.010 [−0.017, −0.005]**	**+0.014 [0.007, 0.020]**	**+0.192 [0.166, 0.218]**

Table 2: HERD minus each baseline after stage 25 on the quantities of Table 1, with 95% paired item-bootstrap intervals (4,000 draws, items resampled within task and G-test category). Bold: the interval excludes zero.

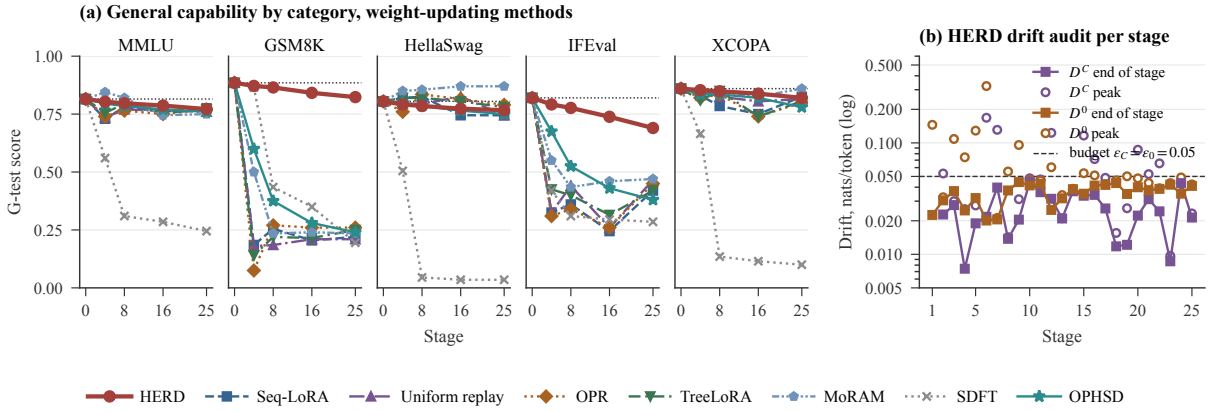


Figure 7: Per-stage view behind Figure 3. (a) G-test score per category at stages 0, 4, 8, 16, and 25 for the weight-updating methods, starting from the base model (dotted). (b) HERD's audited checkpoint drift D^C and base drift D^0 at the end of each stage (filled) and their within-stage peaks (hollow) against the budgets (dashed), on a log axis.

$\varepsilon_C = 0.1$ it stays under 0.1 and is over 0.05 at 15 stages.

Harness, memory, and multipliers by stage. Figure 8a–d follow HERD's state through the stream. The registry gains one skill per task, 24 in all because the stage-7 skill replaced the stage-6 skill under the shared NumGLUE prefix, and ends at 12,982 tokens, 53% of B_K. The memory is at its budget from stage 3 on, within 150 tokens of 24,576; it holds 103 to 204 whole records, 137 at stage 25 from about 16 of the 25 tasks seen. The base multiplier is active at 57% of the updates of stage 1 and at no more than 12% after stage 12; the checkpoint multiplier is never active at 12 stages and at most at 58% of the updates of one stage. The distillation KL falls within every stage. At the first update it is largest on the tasks the base model answers worst, above 2.5 nats per token on Py150, NumGLUE-cm, and SuperNI 024 and 511, and below 0.25 on six of the last nine SuperNI tasks.

Cost by stage. Table 4 gives the training time of every learning method at stage 1, at stage 25, and in total. HERD's stage time follows the task's answer length and not the stage index, 316 s at stage 1 and 284 s at stage 25, as does OPR's, 667 and 705 s; HERD's total of 10,723 s is within 3% of Seq-LoRA's. The two recursive self-distillation baselines grow with the stage, SDFT from 1,999 to 3,626 s as its rollouts lengthen (§3.2) and OPHSD from 11,364 to 30,495 s as it regenerates its teacher context by rollouts. ACE's 888,082 s go to the 38,103 generator, reflector, and curator calls of its offline trainer, and its deployed playbook adds 15,410 tokens to every prompt; HERD deploys bare weights.

Ablations. Table 3 is the full table behind Figure 4, with the SFT objective, the ε_C sweep, and the deployed weights re-scored with the harness. At $\varepsilon_C = 0.02$ the constraint binds: acquisition pays 1.8 points and $\Delta C_{\min}$ improves by 4.5. At $\varepsilon_C = 0.1$ it goes slack: forgetting rises by 1.1 points and the audited D^C ends at 0.071, over 0.05 and under the budget it was given. The smaller moves in the other columns, 0.5 points of forget-

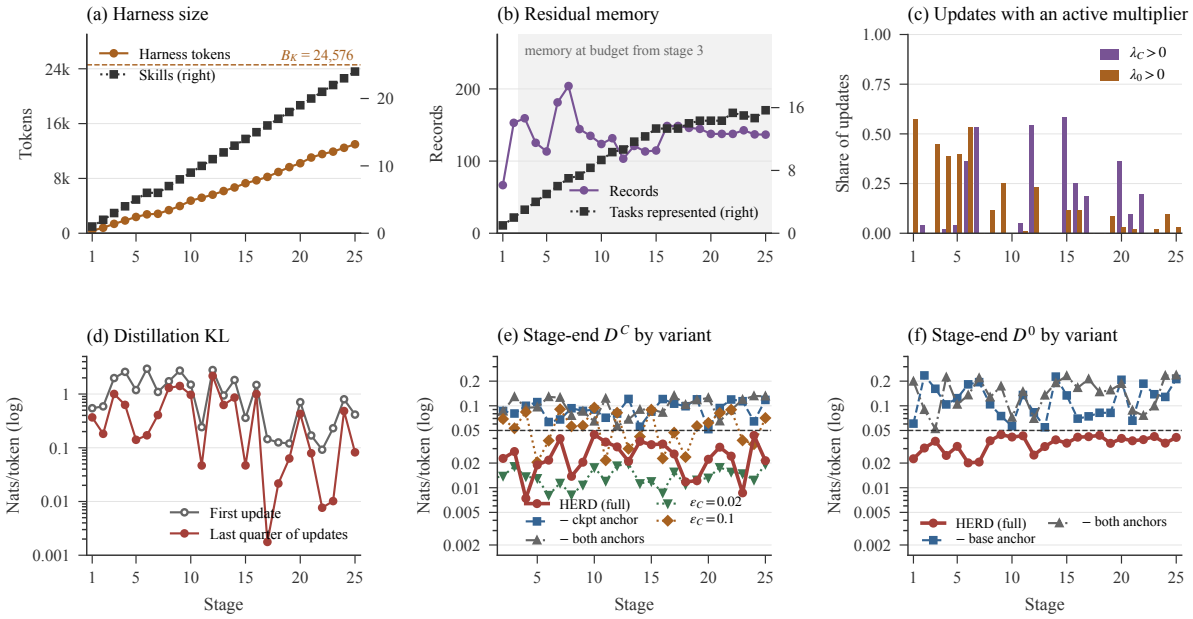


Figure 8: HERD's state by stage; means over three seeds. (a) Registry size in tokens against B_K, and skills in the registry. (b) Residual memory: whole records held and tasks represented; in the shaded stages the memory is at its token budget. (c) Share of a stage's 32 updates at which each multiplier is positive. (d) Distillation KL (Eq. 3) at the first update and averaged over the last eight. (e, f) Audited stage-end D^C and D^0 for full HERD and the single-seed variants of Table 3 that change them; dashed: the 0.05 budget.

Variant	Acq.↑	AA↑	F↓	ΔC ↑	$\Delta C_{\min}$ ↑	D^C	D^0
HERD (full)	0.713±0.002	0.706±0.008	0.010±0.003	−0.063±0.006	−0.130±0.012	0.021	0.041
− skills*	**0.668 (−0.045)**	**0.656 (−0.050)**	0.013 (+0.003)	−0.073 (−0.010)	−0.135 (−0.005)	0.021	0.040
uniform memory*	0.708 (−0.005)	0.690 (−0.017)	0.020 (+0.010)	−0.071 (−0.008)	−0.125 (+0.005)	0.034	0.041
− checkpoint anchor*	0.716 (+0.004)	0.688 (−0.018)	0.030 (+0.020)	−0.075 (−0.012)	−0.140 (−0.010)	**0.118**	0.044
− base anchor*	0.718 (+0.006)	0.704 (−0.002)	0.016 (+0.006)	−0.158 (−0.095)	−0.490 (−0.360)	0.027	**0.212**
− both anchors*	0.721 (+0.008)	0.680 (−0.026)	**0.044 (+0.033)**	**−0.193 (−0.130)**	**−0.560 (−0.430)**	**0.131**	**0.236**
SFT objective*	0.722 (+0.009)	0.706 (−0.001)	0.018 (+0.008)	−0.123 (−0.060)	−0.320 (−0.190)	0.047	0.048
$\varepsilon_C = 0.02$*	0.695 (−0.018)	0.691 (−0.015)	0.005 (−0.005)	−0.051 (+0.012)	−0.085 (+0.045)	0.019	0.038
$\varepsilon_C = 0.1$*	0.717 (+0.004)	0.699 (−0.008)	0.021 (+0.011)	−0.083 (−0.020)	−0.160 (−0.030)	0.071	0.046
+ harness (infer.)‡	0.713 (0.000)	0.708 (+0.002)	0.007 (−0.003)	−0.061 (+0.002)	−0.128 (+0.002)	0.021	0.041

‡ inference only: the deployed weights of the full run re-scored with the stage-25 harness attached to every prompt. * single seed.

Table 3: HERD ablations on TRACE+17, the full table behind Figure 4: scores after stage 25 with the difference from full HERD in parentheses; D^C and D^0 are the audited stage-25 drifts against the budgets $\varepsilon_C = \varepsilon_0 = 0.05$. Bold marks the variant that hurts most in each column and any drift over its budget.

ting at 0.02 and 0.4 of acquisition at 0.1, are inside the seed spread. ε_0 was not swept: removing the base anchor is its $\varepsilon_0 \to \infty$ limit and already costs 9.5 points of ΔC. The last row scores the stage-25 weights with the stage-25 harness prepended to every prompt: AA moves from 0.706 to 0.708, F from 0.010 to 0.007, each G-test category by under 0.2 point, and no task by more than 0.4 points. The results indicate that the model has already internalized the harness.

Seeds and uncertainty. Every number in Tables 1 and 2 and in Figures 3, 7, and 6 is a mean over seeds 42, 43, and 44; the task and G-test cells carry their sample standard deviation at every anchor stage, and every interval is a 95% item bootstrap with 4,000 draws. The ablations and the ε_C sweep are single runs, because the claim is the ordering of the variants and not their level. Three times the full model's displayed seed standard deviations gives approximate reference scales of 0.006 for acquisition, 0.024 for AA, 0.009 for F, 0.018

Method	Training time (s)			Context
	stage 1	stage 25	total	tokens
HERD	316	284	10,723	0
HCL-Stable	580	296	46,887	39
ACE	60,412	36,759	888,082	15,410
Seq-LoRA	51	200	10,463	0
Uniform replay	318	400	15,267	0
OPR	667	705	18,670	0
TreeLoRA	157	256	12,733	0
MoRAM	171	433	14,017	0
SDFT	1,999	3,626	122,508	0
OPHSD	11,364	30,495	659,000	0

Table 4: Training time per stage and in total, and the context attached to every prompt at inference.

for ΔC, and 0.036 for $\Delta C_{\min}$. Gaps within that spread, such as the 0.4-point acquisition change without the checkpoint anchor, are reported in Table 3 and not interpreted.

D Case Studies

HERD against the parametric baselines. Figure 9 follows single test items through the stream. The first two tasks, C-STANCE and FOMC, train on one-letter answers, and 32 cross-entropy updates on them make the short answer a habit: from stage 2 on, the four parametric baselines answer GSM8K with a bare number, median 3 tokens. On the Cho and Chloe item (Figure 9a), the base and HERD write the same three-step derivation to 67 at stages 2 and 25, while Seq-LoRA, uniform replay, OPR, and TreeLoRA guess 10, 11, 14, and 14, then 26, 13, 31, and 22. The behaviours part at the first tokens, where the base anchor looks: in stage 1 of seed 42, one update lifts D^0 to 0.192, λ_0 climbs to 2.5, and the drift is back under 0.05 before the stage ends. At stage 6 (Figure 9b), NumGLUE-cm prompts differ from GSM8K's by one word and demand a bare number. HERD's stage-6 skill says so and is routed to the NumGLUE prefix alone: HERD answers 4320 on the Oliver item and keeps a 211-token chain on the GSM8K item, every parametric baseline answers both with a number, and the base, the static harness, and HCL derive 4320 inside a sentence and score zero.

Retention between the checkpoints. Some losses never reach Table 1. After stage 12, superni:511 with free-text summaries, Seq-LoRA's C-STANCE, FOMC, and ScienceQA fall to 0.03, 0.05, and 0.15 and are back at 0.69, 0.76, and 0.89 by stage 16 (Figure 9c). The knowledge is intact: it answers FOMC with "dovish" or "hawkish" instead of a letter, and mapping the words back gives 0.82, its stage-8 score. HERD's checkpoint probes open with the letter itself, so the same shift registers as D^C: the three seeds peak at 0.114 to 0.136, λ_C rises to 0.76 to 1.99, the stage ends at 0.020 to 0.051, and FOMC stays at 0.75 to 0.80. On MeetingBank, learned at stage 3, HERD's three seeds still write the ordinance title of item 65 at stage 25 (ROUGE-L 0.73) where Seq-LoRA writes "The bill was passed." (0.00), without replaying any earlier answers.

HERD's failing cases. The first limitation is probe coverage. Protection follows the residual memory (Figure 9d,e): a learned task has low residual, leaves the memory, and loses its checkpoint probes with it. Superni:192, copying the sentence that supports a given answer, lost its last probe at stage 17 in all three seeds; seed 42 then fell from 0.83 to 0.52 by stage 25, 70 of its 100 outputs copying the answer already in the input, while seeds 43 and 44 held above 0.90. At stage 25, nine or ten of the 24 earlier tasks have no probe; a floor of probes per learned task is the obvious repair. The second limitation is skill quality. For superni:074, writing a question about a passage, seed 42's skill demands a question and 89 of 100 outputs are questions; the skills of seeds 43 and 44 list a question under Avoid that recurs under Examples, and only 21 and 64 outputs are questions, for ROUGE-L of 0.326 and 0.349 against the base's 0.421. The guard passed all three, as it passed all 75 commits. It checks the window's mean residual without replaying the answer (Eq. 7); improvement under verified-answer prefixes does not guarantee that free generation follows the skill's intended format. The third limitation is probe length. Anchors compare 32 tokens, so on IFEval items with a minimum word count HERD opens like the base, then stops at 232 to 258 words against the base's 499; its pass rate on these 17 items falls from 0.71 to 0.76 at stage 4 to 0.18 to 0.24 at stage 25, against the base's 0.94. Finally, the budget is a mean over a group's probes and tokens, so a few prompts can flip while the stage-end D^0 stays under 0.05: by stage 25, seeds 42 and 43 answer 44 and 60 of the 200 GSM8K items with a bare number, seed 44 only 6, at an accuracy still of 0.81 to 0.83.

OPHSD: drift in wording. OPHSD's outputs stay clean to the end, at most 1.3% cut off, and

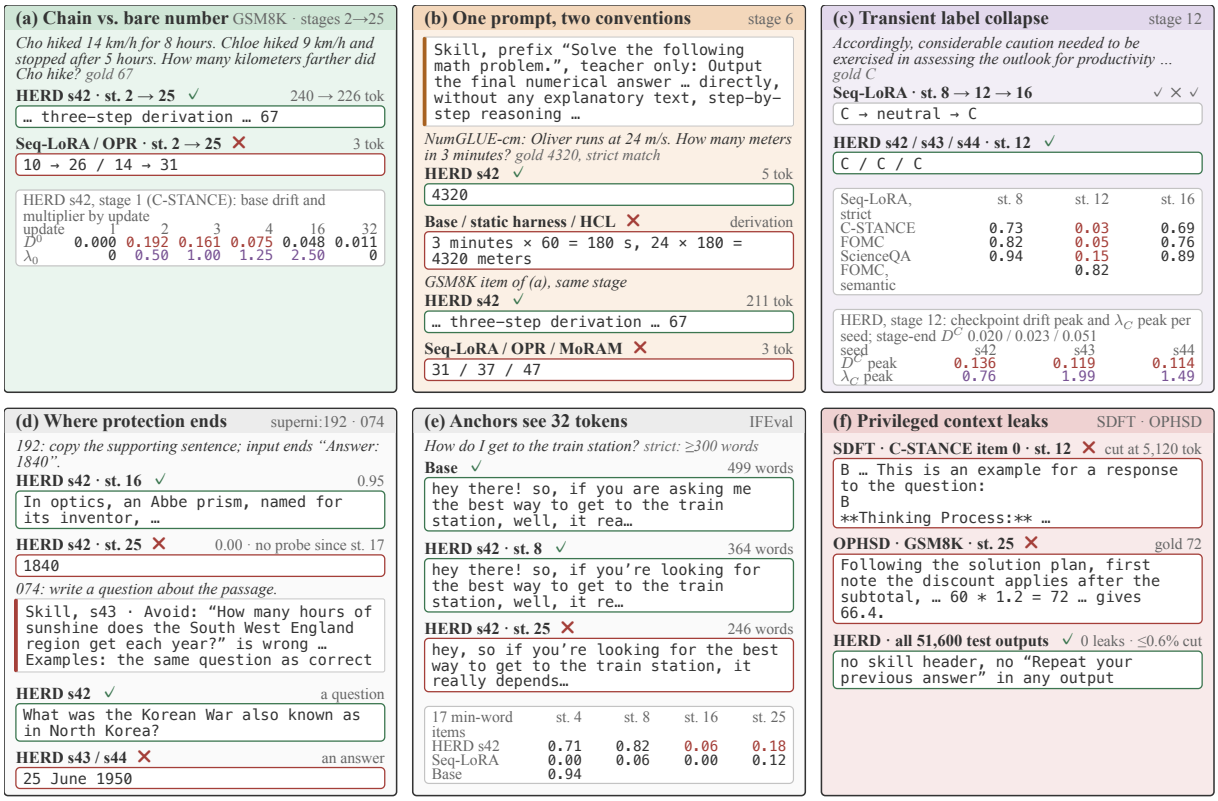


Figure 9: Single test items through the stream; every quoted output is copied from the run records, seed 42 unless marked. (a–c) HERD against the parametric baselines: a GSM8K chain kept through the one-letter stages, with stage 1's base drift and multiplier by update; a bare number on NumGLUE-cm beside a chain on GSM8K at stage 6; Seq-LoRA's label collapse at stage 12, with HERD's checkpoint-drift peaks. (d, e) Where HERD's protection ends: a learned task that lost its probes, a self-contradictory skill, and answer length beyond the 32-token anchor. (f) The privileged teacher context in SDFT's and OPHSD's outputs, and HERD's leak count.

its gains fall on the tasks the base gets right in the wrong format, ScienceQA from 0.03 to 0.85 and NumGLUE from 0 to about 0.5. What drifts is the wording. The official solve message gives the teacher a plan and a "Reference answer (for verification only)", and the student, whose prompt holds neither, begins to cite them (Figure 9f): the share of test outputs that mention a plan or reference answer rises from 0.01 at stage 1 to 0.17 at stage 25, and to 0.84 on the mathematics items, whose outputs shorten from about 1,180 to 405 tokens. Mathematics ends at 0.24 and instruction following at 0.38, against the base's 0.89 and 0.82, with planning-style openers in about 37% of answers against 4% for the base. The harness is run by the current student and the teacher is the stage-start model, so nothing pulls back: the general probe never rises at any of the 25 stages.

SDFT: collapse by leakage and length. SDFT's teacher prompt shows an example and asks for "a response of your own, including the thinking process". Here the example is a one- to three-token label, so the teacher invents a thinking process, and rollouts cut at 1,024 tokens are trained on regardless, so stopping is never supervised. The drift compounds: test outputs cut at the 5,120-token ceiling go from 0.01 at stage 1 to 0.21, 0.74, and 0.94 at stages 4, 8, and 12, and the share that reproduces the teacher's instruction verbatim from 0.00 to 0.23, 0.77, and 0.95. On C-STANCE item 0, stage 12 gives the letter, then "This is an example for a response to the question:" and a new thinking process, restarted until the ceiling (Figure 9f). The loss gives no warning, settling at 0.005 to 0.008, because the teacher is an EMA of the student. HERD's target is the verified answer replayed as the model's own turn, so there is no instruction text to copy, and its KL is scored on that answer's prefix rather than on the student's samples, where teacher and student can agree on a degenerate path.

E Future Works

Larger streams. Our experiments cover one base model and one 25-stage stream; given HERD's for-

getting of 0.010 and fixed per-stage cost (§3.2), we plan to test longer streams with more task families.

Learning from user harnesses. Our long-term goal is a continual learner that trains recursively on a user's own harness, the prompts, skills, and memories they already write (§1), and so internalizes their organization's procedures, domain knowledge, and codebases.

Beyond verified answers and known boundaries. User harnesses supply neither verified answers nor task boundaries. To reduce the dependence on y^*, we are considering: (i) computing the residual and the guard on an accepted target other than y^*, such as a user correction, a rubric, or a majority answer from several harness rollouts; (ii) opening a stage when the mean residual of incoming records rises and closing it when it falls; and (iii) routing skills by retrieval over the record rather than by a template prefix, so that one stage may hold several tasks.

Recursive self-improvement. Because the harness HERD evolves at one stage is distilled into the weights that evolve it at the next, the loop could extend to recursive self-improvement (Zhang et al., 2026b): (i) letting the current checkpoint rather than the frozen θ_0 write the skill (§2.3), so that a model that has internalized one stage writes a better skill for the next; (ii) placing the drift budgets, window size, and record selection in the harness, where the model proposes changes adopted only after a controlled commit; and (iii) applying the same residual test and drift audit to every self-proposed change, with rollback when a gate fails.